

P1468R0

Fixed-layout floating-point type aliases

Published Proposal, 2019-01-21

This version:

<https://wg21.link/P1468R0>

Authors:

[Michał Dominiak](#) (NVIDIA)

[Boris Fomitchev](#) (NVIDIA)

[Sergei Nikolaev](#) (NVIDIA)

Audience:

SG6, EWG, LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1 Abstract

2 Motivation

3 Design

3.1 Supported formats

3.2 Alias naming

3.2.1 `floatXX_t`

3.2.2 `iec559_binaryX_t`

3.2.3 `binaryX_t`

3.2.4 `fp::binaryX_t`

3.2.5 `fp_binaryX_t`

References

Informative References

§ 1. Abstract

This paper proposes a set of `<cstdint>`-style type aliases for floating point types matching specific, well-know floating-point layouts.

This is a companion paper to [\[P1467\]](#). While this paper gives convenient names to types, the other paper allows those names to refer to non-standard, implementation-defined floating-point types.

§ 2. Motivation

The motivation for the general effort of this paper is the same as for [\[P0192\]](#), so we decided to avoid repeating it here, for brevity.

Our previous attempt of introducing floating-point types that match shorter-than-`float` types already provided by multiple software and hardware implementations has been rejected on the grounds of not handling cases where multiple different layouts can be in use by an application at the same time. In our new approach, [\[P1467\]](#) gives us a way to talk about floating-point types other than `float`, `double`

and `long double` within the framework of the standard, while this paper provides convenient names for some of those implementation-defined types.

§ 3. Design

This proposal consists of two parts:

1. Introducing fixed-layout floating-point type aliases.
2. Introducing fixed-layout literals for those aliases.

The first part is what we see as the critical part of this paper, and therefore it is what is discussed by the rest of the paper. The second part is analogous to [\[P1280\]](#), but for the floating-point types defined by the first part. We'd like to hear feedback on whether the second part is also desired before proposing any specifics.

The aliases should receive their own standard library header; our strawman proposal for the name of such a header is `<fixed_float>`. We welcome any naming suggestions that may help avoid spending precious LEWG time bikeshedding the names at a meeting.

Just like the fixed-size integer aliases from `<cstdint>` (`std::int8_t` and family), the aliases proposed here are *optional* if an implementation does not provide a (possibly extended) floating-point type that matches the layout; they are, however, mandatory if such a type exist on an implementation.

§ 3.1. Supported formats

We believe the following layouts should be included:

1. [\[IEEE-754-2008\]](#) `binary16` - half precision floating-point type. Providing this type was the original motivation of [\[P0192\]](#).
2. [\[IEEE-754-2008\]](#) `binary32`.
3. [\[IEEE-754-2008\]](#) `binary64`.
4. [\[IEEE-754-2008\]](#) `binary128`.
5. `bfloat16`, which is `binary32` with 16 bits of precision truncated; see [\[bfloat16\]](#).

`binary32` and `binary64` should be self-explanatory. `binary16` is a format that exists, among others, in LLVM IR; as an optional part of the ARM architecture; as a GCC extension implementing that for ARM; in NVIDIA's CUDA platform. IBM POWER9 has native support for `binary128`. `bfloat16` is a format utilized, for instance, in Google's TPUs, and in TensorFlow.

Question: is the above set of formats sufficient? Are there any other formats that we should include in the first version of this feature?

§ 3.2. Alias naming

We don't know what the best naming scheme here is. We'd like to propose a few options and leave the decision to the committee. Each of these options has its own pros and cons.

§ 3.2.1. `floatXX_t`

1. `std::float16_t`
2. `std::float32_t`
3. `std::float64_t`
4. `std::float128_t`
5. `std::bfloat16_t`

This is the simplest naming scheme of the options presented here; it is also the naming scheme (for `floatXX_t`) used Boost.Math's fixed-layout floating-point types.

On the other hand, nothing in the names of the IEEE aliases implies that they are, in fact, IEEE `binaryXX`. Additionally, `std::float16_t` and `std::bfloat16_t` are slightly too similar for us to feel comfortable about using this set of names.

§ 3.2.2. `iec559_binaryX_t`

1. `std::iec559_binary16_t`
2. `std::iec559_binary32_t`
3. `std::iec559_binary64_t`
4. `std::iec559_binary128_t`
5. `std::bfloat16_t`

This naming scheme is as explicit about the layouts that it guarantees as possible. The two downsides, however, are:

1. these names (short of `std::bfloat16_t`) are *long*, and

2. most people don't recognize `iec559` as quickly as they'd recognize `ieee754`; `iec559`, however, is how we refer to [\[IEEE-754-2008\]](#) in the rest of the library, so if we go this path, we should probably stay consistent with the rest of the language.

§ 3.2.3. `binaryX_t`

1. `std::binary16_t`
2. `std::binary32_t`
3. `std::binary64_t`
4. `std::binary128_t`
5. `std::bfloat16_t`

These names are shorter than `std::iec559_binaryX_t`, but they are less obvious; nothing in their names directly points at them being floating-point types. On the other hand, when a user of the language knows that these are floating-point types, they should recognize them as names of [\[IEEE-754-2008\]](#) formats.

§ 3.2.4. `fp::binaryX_t`

1. `std::fp::binary16_t`
2. `std::fp::binary32_t`
3. `std::fp::binary64_t`
4. `std::fp::binary128_t`
5. `std::fp::bfloat16_t`

The short, nested namespace within `std` should make it more obvious that these types are floating-point types; from there, recognizing `binary16` as a name of a specific [\[IEEE-754-2008\]](#) format is much easier. Users who know what the `binaryX_t` types are can just import the contents of the entire namespace, to avoid having to repeat `std::fp` everywhere.

The cons of this approach are:

1. it introduces a namespace that doesn't serve a `_big_` purpose; and
2. the alias for `bfloat16` already includes `float` in its name, which means that `std::fp::bfloat16_t` is somewhat redundant.

§ 3.2.5. `fp_binaryX_t`

1. `std::fp_binary16_t`
2. `std::fp_binary32_t`
3. `std::fp_binary64_t`
4. `std::fp_binary128_t`
5. `std::fp_bfloat16_t`

This is a slight modification of the previous scheme, which trades the nested namespace within `std::` for an `fp_` prefix within the name. This makes it harder to stop repeating the `fp_` part, because you can no longer just import a namespace that includes all of these.

§ References

§ Informative References

[BFLOAT16]

[bfloat16 floating-point format](https://en.wikipedia.org/wiki/Bfloat16_floating-point_format). URL: https://en.wikipedia.org/wiki/Bfloat16_floating-point_format

[IEEE-754-2008]

[IEEE Standard for Floating-Point Arithmetic](http://ieeexplore.ieee.org/servlet/opac?punumber=4610933). 29 August 2008. URL: <http://ieeexplore.ieee.org/servlet/opac?punumber=4610933>

[P0192]

Michał Dominiak; et al. [`short float` and fixed-size floating point types](https://wg21.link/P0192). URL: <https://wg21.link/P0192>

[P1280]

Isabella Muerte. [Integer Width Literals](https://wg21.link/P1280). URL: <https://wg21.link/P1280>

[P1467]

Michał Dominiak; David Olsen. [Extended floating-point types](https://wg21.link/P1467). URL: <https://wg21.link/P1467>